Eduvos Assessment Coversheet

Academic Details

Campus: **Online**

Faculty: **Information Technology**

Module Code: **ITNPA4-34**

Lecturer's Name: **Tinotenda Mavetera**

Group (if applicable) **1**

Assessment Details

Assessment Type: **Individual Assessment**

Assignment/Brief Number:

Due Date/ Submission Date: **13 November 2026**

Student Details (Group, if applicable)

#	Full Name	Student Number	Contact Number	% Participation	Signature
1					
2					
3					
4					
5					
6					

Student Details (Individual, if applicable)

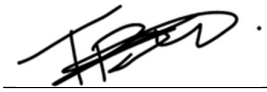
Student Full Name: **Thabo Baleni**

Student Number: **EDUV4818947**

Academic Integrity

AI Disclosure Completed	[X] **Yes**	[ ] **No**

Declaration:
I declare that this assessment is my own original work except for source material explicitly acknowledged. I confirm that this work has not previously been submitted for assessments in any course. I acknowledge the institutional AI policy and confirm that AI tools have only been used as stipulated and documented in the AI Disclosure Appendix.

Student Signature 

Date **01 November 2026**

Lecturer Use Only

Lecturer Comments:

Marks Awarded (%):

Lecturer Signature:

Date:

Table of Contents

Eduvos Assessment Coversheet 1
Assessment 4
Question 1 4
1.1) 4
1.2) 8
1.3) 11
Question 2 18
2.1) 18
2.2) 22
2.3) 24
Question 3 29
3.1) 29
3.2) 30
3.3) 31
3.4) 35
Question 4 40
4.1) 40
4.2) 42
AI Evidence Section 44
AI Disclosure Appendix 45
Declaration: 45

Assessment

GitHub link

https://github.com/Thabo-4818947/ITNPA4-34---Individual-Assessment---Online---EDUV4818947

Question 1

1.1)

```
import pandas as pd
import json
import warnings

warnings.filterwarnings('ignore')

with open('research_abstracts.json', 'r', encoding='utf-8') as file:
    data = json.load(file)

# JSON data into a DataFrame
df = pd.DataFrame(data)

df.head()

# Dataset dimensions
print("Dataset shape:", df.shape)

# Column names and data types
print("\nDataset information:")
df.info()
```

```
# Missing values
print("\nMissing values:")
print(df.isnull().sum())

# Duplicate records
print("\nDuplicate records:", df.duplicated().sum())

# Distribution of research categories
print("\nLabel distribution:")
print(df['Label'].value_counts())

# Removing duplicate records
df = df.drop_duplicates()

# Resetting the DataFrame index
df = df.reset_index(drop=True)

print("Dataset shape after removing duplicates:", df.shape)
print("Remaining duplicate records:", df.duplicated().sum())

import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Text denoising techniques
# NLTK resources
nltk.download('punkt')
```

```
nltk.download('punkt_tab')
nltk.download('stopwords')


stop_words = set(stopwords.words('english'))


stop_words = stop_words - {'no', 'not', 'nor'}

# Text preprocessing function
def preprocess(text):


    clean_text = text.lower()


    clean_text = re.sub(r'[^a-zA-Z\s]', ' ', clean_text)


    clean_text = re.sub(r'\s+', ' ', clean_text).strip()


    words = word_tokenize(clean_text)


    filtered_words = [
        word for word in words
        if word not in stop_words
    ]
```

```
    clean_text = ' '.join(filtered_words)

    return clean_text


# Preprocessing to the Abstract column
df['Clean_Abstract'] = df['Abstract'].apply(preprocess)

# Original and cleaned abstracts
df[['Abstract', 'Clean_Abstract', 'Label']].head()

print("Original abstract:")
print(df['Abstract'].iloc[0])

print("\nCleaned abstract:")
print(df['Clean_Abstract'].iloc[0])

# The length of the original and cleaned abstracts
df['Original_Length'] = df['Abstract'].str.len()
df['Cleaned_Length'] = df['Clean_Abstract'].str.len()

print("\nAverage original abstract length:",
      df['Original_Length'].mean())

print("Average cleaned abstract length:",
      df['Cleaned_Length'].mean())
```

```
print("\nFinal dataset shape:", df.shape)
df.head()
```

1.2)

```
# Input feature
texts = df['Clean_Abstract'].astype(str).values

# Target variable
labels = df['Label'].values

# Selected features
print("Number of abstracts:", len(texts))
print("Number of labels:", len(labels))

print("\nInput feature example:")
print(texts[0][:500])

print("\nTarget label example:")
print(labels[0])

import numpy as np

# Categorical labels into numerical values
label_mapping = {
    'classical': 0,
    'ai': 1
}
```

```
y = np.array([label_mapping[label] for label in labels])

print("Label mapping:", label_mapping)

print("\nEncoded labels:")
print(y[:10])

print("\nClass distribution:")
print(pd.Series(y).value_counts())

from sklearn.model_selection import train_test_split

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    texts,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

print("\nTraining class distribution:")
print(pd.Series(y_train).value_counts())

print("\nTesting class distribution:")
print(pd.Series(y_test).value_counts())
```

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Feature parameters
max_features = 10000
maxlen = 500

# Tokenizer
tokenizer = Tokenizer(
    num_words=max_features,
    oov_token="<OOV>"
)


tokenizer.fit_on_texts(X_train)

# Converting text into numerical sequences
X_train_sequences = tokenizer.texts_to_sequences(X_train)
X_test_sequences = tokenizer.texts_to_sequences(X_test)

# Padding
X_train_padded = pad_sequences(
    X_train_sequences,
    maxlen=maxlen,
    padding='post',
    truncating='post'
)
```

```
X_test_padded = pad_sequences(
    X_test_sequences,
    maxlen=maxlen,
    padding='post',
    truncating='post'
)



print("Training feature shape:", X_train_padded.shape)
print("Testing feature shape:", X_test_padded.shape)

print("\nNumber of unique words:", len(tokenizer.word_index))

# Numerical sequences
print("Original cleaned abstract:")
print(X_train[0][:300])

print("\nNumerical sequence:")
print(X_train_sequences[0][:30])

print("\nPadded sequence:")
print(X_train_padded[0][:30])

print("\nTraining target:")
print(y_train[0])
```

1.3)

```
import numpy as np
import matplotlib.pyplot as plt
```

```
import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

# Random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)


model = Sequential()


model.add(tf.keras.Input(shape=(maxlen,)))

# Embedding layer
model.add(Embedding(
    input_dim=max_features,
    output_dim=32
))
```

```
# LSTM layer
model.add(LSTM(
    128,
    dropout=0.2,
    recurrent_dropout=0.2
))

# Output layer
model.add(Dense(
    1,
    activation='sigmoid'
))

# Compile model
model.compile(
    optimizer='rmsprop',
    loss='binary_crossentropy',
    metrics=['accuracy']
)



model.summary()

# Training the LSTM model
history = model.fit(
    X_train_padded,
    y_train,
    epochs=10,
    batch_size=60,
```

```
    validation_split=0.2,
    verbose=1
)

# Performance history
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)

# Training and validation accuracy
plt.figure(figsize=(10, 5))

plt.plot(epochs, acc, label='Training Accuracy')
plt.plot(epochs, val_acc, label='Validation Accuracy')

plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()

# Training and validation loss
plt.figure(figsize=(10, 5))
```

```
plt.plot(epochs, loss, label='Training Loss')
plt.plot(epochs, val_loss, label='Validation Loss')

plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()

# The model on the test dataset
test_loss, test_accuracy = model.evaluate(
    X_test_padded,
    y_test,
    verbose=1
)

print("Test Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_accuracy, 4))
print("Test Accuracy (%):", round(test_accuracy * 100, 2), "%")



# Prediction probabilities
y_pred_probability = model.predict(X_test_padded)

# Probabilities into binary predictions
y_pred = (y_pred_probability >= 0.5).astype(int).ravel()

# Accuracy
```

```
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))

# Classification report
print("\nClassification Report:\n")

print(classification_report(
    y_test,
    y_pred,
    target_names=['Classical', 'AI'],
    zero_division=0
))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Confusion matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['Classical', 'AI']
)

disp.plot(cmap='Blues', values_format='d')

plt.title('LSTM Baseline Model - Confusion Matrix')
plt.show()

print("Confusion Matrix:")
print(cm)
```

```
# Final training and validation metrics
final_train_accuracy = history.history['accuracy'][-1]
final_val_accuracy = history.history['val_accuracy'][-1]

final_train_loss = history.history['loss'][-1]
final_val_loss = history.history['val_loss'][-1]

# Performance summary
performance = pd.DataFrame({
    'Metric': [
        'Training Accuracy',
        'Validation Accuracy',
        'Test Accuracy',
        'Training Loss',
        'Validation Loss',
        'Test Loss'
    ],
    'Value': [
        final_train_accuracy,
        final_val_accuracy,
        test_accuracy,
        final_train_loss,
        final_val_loss,
        test_loss
    ]
})

display(performance)
```

Question 2

2.1)

```
import json
import re
from pathlib import Path
from importlib.metadata import version

import nltk
import pandas as pd
import joblib
from nltk.corpus import stopwords

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
)
```

```
nltk.download("punkt", quiet=True)
nltk.download("stopwords", quiet=True)
nltk.download("punkt_tab", quiet=True)

with open("research_abstracts.json", "r", encoding="utf-8") as file:
    data = json.load(file)

df = pd.DataFrame(data)

print("Original dataset shape:", df.shape)
display(df.head())

# Cleaning text and spliting data
df = df.drop_duplicates().reset_index(drop=True)
df["Label"] = df["Label"].str.strip().str.lower()

stop_words = set(stopwords.words("english")) - {"no", "not", "nor"}

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"[^a-zA-Z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()

    tokens = nltk.word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words]

    return " ".join(tokens)

df["Clean_Abstract"] = df["Abstract"].apply(clean_text)
```

```
df["Label_Encoded"] = df["Label"].map({"classical": 0, "ai": 1})

if df["Label_Encoded"].isna().any():
    raise ValueError("Some labels were not recognised. Check the Label column.")

X = df["Clean_Abstract"]
y = df["Label_Encoded"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)

print("Dataset shape after removing duplicates:", df.shape)
print("\nClass distribution:")
print(df["Label"].value_counts())
print("\nTraining records:", len(X_train))
print("Testing records:", len(X_test))
print("\nMissing cleaned abstracts:", X.isna().sum())
print("\nSample cleaned abstract:")
print(X.iloc[0])

#2.1 Training four models
models = {
    "Multinomial Naive Bayes": MultinomialNB(),
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
```

```
    "Support Vector Machine": LinearSVC(random_state=42),
    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        random_state=42,
        n_jobs=-1,
    ),
}

results = []
trained_models = {}

for name, classifier in models.items():
    pipeline = Pipeline([
        ("tfidf", TfidfVectorizer(
            max_features=20000,
            ngram_range=(1, 2),
            sublinear_tf=True,
        )),
        ("classifier", classifier),
    ])

    pipeline.fit(X_train, y_train)
    predictions = pipeline.predict(X_test)
    trained_models[name] = pipeline

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, predictions),
```

```
        "AI Precision": precision_score(y_test, predictions, pos_label=1,
zero_division=0),
        "AI Recall": recall_score(y_test, predictions, pos_label=1, zero_division=0),
        "AI F1-Score": f1_score(y_test, predictions, pos_label=1, zero_division=0),
    })

    print(f"\n{'=' * 60}\n{name}\n{'=' * 60}")
    print(classification_report(
        y_test,
        predictions,
        labels=[0, 1],
        target_names=["Classical", "AI"],
        zero_division=0,
    ))
    print("Confusion matrix:")
    print(confusion_matrix(y_test, predictions, labels=[0, 1]))

results_df = pd.DataFrame(results).sort_values(
    by="AI F1-Score",
    ascending=False,
).reset_index(drop=True)

print("\nModel performance comparison:")
display(results_df)
```

2.2)

```
#2.2 Evaluating balanced SVM
balanced_svm = Pipeline([
    ("tfidf", TfidfVectorizer(
```

```
        max_features=20000,
        ngram_range=(1, 2),
        sublinear_tf=True,
    )),
    ("classifier", LinearSVC(
        class_weight="balanced",
        random_state=42,
    )),
])

balanced_svm.fit(X_train, y_train)
balanced_predictions = balanced_svm.predict(X_test)

original_svm = trained_models["Support Vector Machine"]
original_predictions = original_svm.predict(X_test)

comparison_df = pd.DataFrame([
    {
        "Model": "Original SVM",
        "Accuracy": accuracy_score(y_test, original_predictions),
        "AI Precision": precision_score(y_test, original_predictions, pos_label=1,
zero_division=0),
        "AI Recall": recall_score(y_test, original_predictions, pos_label=1,
zero_division=0),
        "AI F1-Score": f1_score(y_test, original_predictions, pos_label=1,
zero_division=0),
    },
    {
        "Model": "Enhanced SVM - Balanced Class Weights",
        "Accuracy": accuracy_score(y_test, balanced_predictions),
```

```
        "AI Precision": precision_score(y_test, balanced_predictions, pos_label=1,
zero_division=0),
        "AI Recall": recall_score(y_test, balanced_predictions, pos_label=1,
zero_division=0),
        "AI F1-Score": f1_score(y_test, balanced_predictions, pos_label=1,
zero_division=0),
    },
])


display(comparison_df)


print("\nEnhanced SVM classification report:")
print(classification_report(
    y_test,
    balanced_predictions,
    labels=[0, 1],
    target_names=["Classical", "AI"],
    zero_division=0,
))


print("Enhanced SVM confusion matrix:")
print(confusion_matrix(y_test, balanced_predictions, labels=[0, 1]))
```

2.3)

```
#2.3 Algorithmic configuration
# Saving the model
import joblib


deployment_folder = Path("deployment")
deployment_folder.mkdir(exist_ok=True)
```

```
model_path = deployment_folder / "balanced_svm_model.pkl"
joblib.dump(balanced_svm, model_path)

print(f"Model saved to: {model_path.resolve()}")

%%writefile deployment/classifier.py
import re
from pathlib import Path

import joblib
import nltk
import streamlit as st
from nltk.corpus import stopwords

nltk.download("stopwords", quiet=True)
STOP_WORDS = set(stopwords.words("english")) - {"no", "not", "nor"}

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"[^a-zA-Z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    tokens = text.split()
    tokens = [word for word in tokens if word not in STOP_WORDS]
    return " ".join(tokens)

st.set_page_config(page_title="Abstract Classifier", page_icon=" 📄 ")
```

```
model_path = Path(__file__).parent / "balanced_svm_model.pkl"
model = joblib.load(model_path)

st.title("Research Abstract Classifier")
st.write("Classify a research abstract as AI-generated or Classical.")

abstract = st.text_area(
  "Paste the research abstract below:",
  height=250,
  placeholder="Enter the abstract text here...",
)

if st.button("Classify"):
  if not abstract.strip():
    st.warning("Please enter an abstract first.")
  else:
    cleaned_abstract = clean_text(abstract)
    prediction = int(model.predict([cleaned_abstract])[0])

    if prediction == 1:
      st.subheader("Prediction: AI")
    else:
      st.subheader("Prediction: Classical")

#Requirements and readme
requirements = [
  f"streamlit=={version('streamlit')}",
  f"scikit-learn=={version('scikit-learn')}",
```

```
    f"joblib=={version('joblib')}",
    f"nltk=={version('nltk')}",
]

(deployment_folder / "requirements.txt").write_text(
    "\n".join(requirements) + "\n",
    encoding="utf-8",
)

readme_lines = [
    "# Research Abstract Classifier",
    "",
    "A Streamlit app that classifies research abstracts as AI-generated or Classical using a balanced SVM model.",
    "",
    "## Files",
    "",
    "- `classifier.py` — Streamlit web app",
    "- `balanced_svm_model.pkl` — fitted TF-IDF and balanced SVM pipeline",
    "- `requirements.txt` — package versions used by the app",
    "",
    "## Run locally",
    "",
    "Open Anaconda Prompt from the folder containing `deployment`, then run:",
    "",
    "```bash",
    "cd deployment",
    "pip install -r requirements.txt",
```

```
    "streamlit run classifier.py",
    "```",
]

(deployment_folder / "README.md").write_text(
    "\n".join(readme_lines) + "\n",
    encoding="utf-8",
)

print("Created requirements.txt and README.md in the deployment folder.")

!streamlit run deployment/classifier.py --server.port 8502
```

Question 3

3.1)

```
from pathlib import Path, PurePosixPath

import zipfile

import pandas as pd


zip_path = Path(

  r"C:\Users\Tbose\Downloads\Project 1 Required Resources-
20261102\research_corpus.zip"

)


with zipfile.ZipFile(zip_path, "r") as archive:

  text_files = sorted(

    name for name in archive.namelist()

    if name.lower().endswith(".txt")

  )


  articles = []

  for name in text_files:

    content = archive.read(name).decode("utf-8", errors="replace")

    articles.append({

      "article_id": PurePosixPath(name).stem,

      "filename": name,

      "content": content,

    })


articles_df = pd.DataFrame(articles)
```

```
print("Articles loaded:", len(articles_df))
print("Dataframe shape:", articles_df.shape)
display(articles_df[["article_id", "filename"]])
print("\nFirst article preview:")
print(articles_df.loc[0, "content"][:500])
```

3.2)

```
# 3.2
import re
import unicodedata
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

nltk.download("punkt", quiet=True)
nltk.download("stopwords", quiet=True)
nltk.download("punkt_tab", quiet=True)

stop_words = set(stopwords.words("english"))

def denoise_layout(text):
    text = unicodedata.normalize("NFKC", str(text))
    text = text.replace("\r\n", "\n").replace("\r", "\n")
    text = re.sub(r"[\x00-\x08\x0b\x0c\x0e-\x1f\x7f]", " ", text)
    text = re.sub(r"(?im)^[ \t]*(?:page[ \t]*)?\d+[ \t]*\n?", "", text)
    text = re.sub(r"(?<=\w)-[ \t]*\n[ \t]*(?=[a-z])", "", text)
    text = re.sub(r"(?<!\n)\n(?!\n)", " ", text)
    text = re.sub(r"[ \t]+", " ", text)
```

```
    text = re.sub(r" *\n *", "\n", text)
    text = re.sub(r"\n{3,}", "\n\n", text)
    return text.strip()

def prepare_tokens(text):
    words = word_tokenize(text.lower())
    return [
        word for word in words
        if word.isalpha() and word not in stop_words
    ]

articles_df["denoised_content"] = articles_df["content"].apply(denoise_layout)
articles_df["tokens"] = articles_df["denoised_content"].apply(prepare_tokens)
articles_df["preprocessed_content"] = articles_df["tokens"].apply(" ".join)

print("Articles processed:", len(articles_df))
display(
    articles_df.assign(
        token_count=articles_df["tokens"].apply(len)
    )[["article_id", "token_count"]]
)
print("\nFirst article token preview:")
print(articles_df.loc[0, "tokens"][:40])
```

3.3)

```
import numpy as np
import matplotlib.pyplot as plt
from nltk.tokenize import sent_tokenize
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

first_article = articles_df.loc[0, "denoised_content"]

sentences = [
   sentence.strip()
   for sentence in sent_tokenize(first_article)
   if len(sentence.split()) > 4
]

vectorizer = TfidfVectorizer(stop_words="english")
sentence_vectors = vectorizer.fit_transform(sentences)
similarity = cosine_similarity(sentence_vectors)

# Sentence-similarity graph
np.fill_diagonal(similarity, 0)
row_sums = similarity.sum(axis=1, keepdims=True)

transition = np.divide(
   similarity,
   row_sums,
   out=np.zeros_like(similarity),
   where=row_sums != 0,
)


dangling_rows = np.where(row_sums.ravel() == 0)[0]
```

```
transition[dangling_rows, :] = 1 / len(sentences)

# TextRank sentence scores
scores = np.full(len(sentences), 1 / len(sentences))
damping = 0.85

for _ in range(100):
  updated_scores = (
    (1 - damping) / len(sentences)
    + damping * transition.T.dot(scores)
  )

  if np.allclose(scores, updated_scores, atol=1e-6):
    scores = updated_scores
    break

  scores = updated_scores

# Highest-ranked sentences
ranked_indices = np.argsort(scores)[::-1]
selected_indices = []
similarity_threshold = 0.65

for index in ranked_indices:
  if all(
    similarity[index, selected] < similarity_threshold
    for selected in selected_indices
  ):
```

```
        selected_indices.append(int(index))

    if len(selected_indices) == 5:
        break


summary_indices = sorted(selected_indices)
first_article_summary = " ".join(
    sentences[index] for index in summary_indices
)

print("TextRank extractive summary:")
print(first_article_summary)
print(f"\nOriginal words: {len(first_article.split())}")
print(f"Summary words: {len(first_article_summary.split())}")
print(f"Unique sentences selected: {len(summary_indices)}")

# Visualize the selected sentences' TextRank scores
plot_indices = sorted(selected_indices, key=lambda index: scores[index])

plt.figure(figsize=(8, 4))
plt.barh(
    [f"Sentence {index + 1}" for index in plot_indices],
    [scores[index] for index in plot_indices],
    color="steelblue",
)
plt.xlabel("TextRank importance score")
plt.title("Selected sentences in research_1")
```

```
plt.tight_layout()
plt.show()
```

3.4)

```
import re
from collections import Counter

import matplotlib.pyplot as plt
from nltk.tokenize import sent_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer

remaining_articles = articles_df.iloc[1:].copy()

def is_complete_sentence(sentence):
    sentence = sentence.strip()
    word_count = len(sentence.split())

    ends_with_punctuation = bool(
        re.search(r'[.!?]["’”\')\]]*$', sentence)
    )
    balanced_parentheses = sentence.count("(") == sentence.count(")")
    balanced_brackets = sentence.count("[") == sentence.count("]")

    return (
        12 <= word_count <= 45
        and sentence[:1].isupper()
        and ends_with_punctuation
```

```
        and balanced_parentheses
        and balanced_brackets
    )

sentence_records = []

for _, row in remaining_articles.iterrows():
    for sentence_number, sentence in enumerate(
        sent_tokenize(row["denoised_content"])
    ):
        sentence = sentence.strip()
        if is_complete_sentence(sentence):
            sentence_records.append({
                "article_id": row["article_id"],
                "sentence_number": sentence_number,
                "sentence": sentence,
            })

if not sentence_records:
    raise ValueError("No complete sentences passed the filters.")

sentences_3_4 = [record["sentence"] for record in sentence_records]

tfidf_vectorizer = TfidfVectorizer(
    stop_words="english",
    ngram_range=(1, 2),
)
sentence_matrix = tfidf_vectorizer.fit_transform(sentences_3_4)
```

```
sentence_scores = []
for row_number in range(sentence_matrix.shape[0]):
    row = sentence_matrix.getrow(row_number)
    sentence_scores.append(row.data.mean() if row.nnz else 0)

# Two high-ranking, non-redundant sentences per article
aggregate_summary = []

for article_id in remaining_articles["article_id"]:
    candidate_indices = [
        i for i, record in enumerate(sentence_records)
        if record["article_id"] == article_id
    ]
    candidate_indices.sort(
        key=lambda i: sentence_scores[i],
        reverse=True,
    )

    selected_for_article = []

    for index in candidate_indices:
        is_redundant = any(
            sentence_matrix[index]
            .multiply(sentence_matrix[selected])
            .sum() >= 0.55
            for selected in selected_for_article
        )
```

```
        if not is_redundant:
            selected_for_article.append(index)

        if len(selected_for_article) == 2:
            break

    for index in sorted(
        selected_for_article,
        key=lambda i: sentence_records[i]["sentence_number"],
    ):
        aggregate_summary.append(sentence_records[index])

print("Aggregate TF-IDF extractive summary:")
for record in aggregate_summary:
    print(f"\n[{record['article_id']}] {record['sentence']}")

generic_terms = {
    "research", "study", "article", "one", "use", "using",
    "also", "however", "result", "results",
}
term_counts = Counter(
    token
    for tokens in remaining_articles["tokens"]
    for token in tokens
    if token not in generic_terms
)
```

```
top_terms = term_counts.most_common(15)

if top_terms:
    terms = [term for term, _ in top_terms][::-1]
    counts = [count for _, count in top_terms][::-1]

    plt.figure(figsize=(9, 6))
    plt.barh(terms, counts, color="darkcyan")
    plt.xlabel("Frequency across articles")
    plt.title("Key terms in research_2 to research_5")
    plt.tight_layout()
    plt.show()

print(f"\nSentences in aggregate summary: {len(aggregate_summary)}")
```

Question 4

4.1)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

n_topics = 3



count_vectorizer = CountVectorizer(
   max_features=1000,
   max_df=0.95,
   min_df=1,
)

document_term_matrix = count_vectorizer.fit_transform(
   articles_df["preprocessed_content"]
)

lda_model = LatentDirichletAllocation(
   n_components=n_topics,
   random_state=42,
   learning_method="batch",
   max_iter=20,
)
```

```
document_topic_distribution = lda_model.fit_transform(document_term_matrix)
feature_names = count_vectorizer.get_feature_names_out()

# Top words for each topic
for topic_number, topic_weights in enumerate(lda_model.components_, start=1):
    top_word_indices = topic_weights.argsort()[::-1][:10]
    top_words = [feature_names[i] for i in top_word_indices]
    print(f"Topic {topic_number}: {', '.join(top_words)}")

# Each article's topic proportions
topic_columns = [f"Topic {i}" for i in range(1, n_topics + 1)]
topic_distribution_df = pd.DataFrame(
    document_topic_distribution,
    columns=topic_columns,
)
topic_distribution_df.insert(0, "article_id", articles_df["article_id"].values)

print("\nTopic distribution by article:")
display(topic_distribution_df.round(3))

# Topic proportions as a stacked bar chart
x = np.arange(len(articles_df))
bottom = np.zeros(len(articles_df))

plt.figure(figsize=(9, 5))
for topic_index in range(n_topics):
    values = document_topic_distribution[:, topic_index]
```

```
    plt.bar(
        x,
        values,
        bottom=bottom,
        label=f"Topic {topic_index + 1}",
    )
    bottom += values

plt.xticks(x, articles_df["article_id"], rotation=0)
plt.ylabel("Topic proportion")
plt.xlabel("Article")
plt.title("LDA topic distribution by article")
plt.ylim(0, 1)
plt.legend()
plt.tight_layout()
plt.show()
```

4.2)

```
from pathlib import Path
import pyLDAvis
import pyLDAvis.lda_model
from IPython.display import display

lda_visualization = pyLDAvis.lda_model.prepare(
    lda_model,
    document_term_matrix,
    count_vectorizer,
)
```

```
display(pyLDAvis.display(lda_visualization))


output_folder = Path("outputs")
output_folder.mkdir(exist_ok=True)


html_path = output_folder / "lda_intertopic_distance.html"
pyLDAvis.save_html(lda_visualization, str(html_path))


print(f"Interactive map saved to: {html_path.resolve()}")
```

4.3)

GitHub link

AI Evidence Section

[AI Code 1 – 4 may necessitate evidence submission as specified in your assessment instruction, this section is reserved for evidences where relevant, please delete if not applicable.]

AI Disclosure Appendix

AI tool used	Purpose / how it was used	How I verified or adapted the output
ChatGPT	Natural Language Processing research	Cross-checked with Eduvos notes and textbook material.
ChatGPT	Jupyter notebook Streamlit setup	Cross-checked and internet-searched sources on Streamlit to verify if ChatGPT output was the correct use for the setup
Claude	LDA uses and other examples of text processing, text modelling, and text classification	Cross-checked and internet-searched sources on text processing and text coding sources to verify that Claude's output was valid and correct. Used the output examples as a guide to complete the task in the Assignment

Declaration:

I/we declare that this assessment is my/our own original work, except for source material explicitly acknowledged. I/we confirm that any use of Artificial Intelligence (AI) tools has been applied only as permitted by the assessment instructions and has been fully disclosed where required.

I/we understand that I/we remain responsible for the accuracy, originality, and academic quality of this submission. I/we am/are aware of the institution's policies on academic integrity and the requirements relating to the responsible use of AI in assessments.

I/we acknowledge that failure to comply with these requirements, including undeclared or inappropriate AI use, may result in academic penalties or disciplinary procedures in accordance with institutional policy.

Student Signature(s)	Date
T.Baleni	**01 November 2026**